



Make Dictionary from Elements at Given Index

Question 3: 2025 Jan OPPE 1 - Set 1

The Problem

Write a function that creates a dictionary from single elements at a specified index

Function Signature

```
make_dict_from_elems_in_index(keys, values, index)
```

Input: List of keys, list of values, and an integer index

Output: Dictionary with one key-value pair

Key =

Element from `keys` at `index`

Value =

Element from `values` at `index`



The index is always valid. No input validation or printing needed—just complete the function.

Understand What Is Actually Required

✗ What This is NOT

- Combining entire lists together
- Creating a complete dictionary with all pairs
- Iterating through multiple elements
- Searching or finding an index

✓ What This IS

- A single key-value pair only
- Direct access using known index
- One element from each list
- Very simple, elegant solution

The key insight: You're given the **exact position** you need. No searching required.

Break It Down with Examples

```
keys = ['apple', 'banana', 'cherry']  
values = [10, 20, 30, 40]
```

01

index = 1

keys[1] »→ 'banana'

values[1] »→ 20

Result: {'banana': 20}

02

index = 0

keys[0] »→ 'apple'

values[0] »→ 10

Result: {'apple': 10}

03

index = -1 (Negative Indexing)

keys[-1] »→ 'cherry'

values[-1] »→ 40

Result: {'cherry': 40}

Python handles negative indexing automatically. -1 means the last element, -2 means second to last, and so on.

The Core Concept: List Indexing

Direct Access

Access any element using its position:

```
keys[index]  
values[index]
```

No Loops Needed

Since you know the exact location, there's no need for iteration or searching through the list.

No Conditions Required

The problem guarantees the index is valid, so you don't need to check bounds or add error handling.

Dictionary Creation Syntax

1

The Pattern

```
{key : value}
```

2

With Variables

```
{keys[index] : values[index]}
```

3

Complete Solution

```
return {keys[index]:  
        values[index]}
```

Dictionaries in Python are created using curly braces `{}` with a colon between key and value.

That's the entire logic. Place the indexed elements directly into the dictionary syntax and return it.

The Elegant Solution

One Line

No loop. No temporary variable. No conditions. Just pure, direct indexing.

```
def make_dict_from_elems_in_index(  
    keys,  
    values,  
    index: int  
) -> dict:  
    return {keys[index]: values[index]}
```

This is Pythonic code: simple, readable, and efficient. The function takes three parameters and returns a dictionary with one key-value pair created from the elements at the specified index in both lists.

Common Pitfall: Overthinking

1

The Overcomplicated Approach

```
d = {}  
for i in range(len(keys)):  
    if i == index:  
        d[keys[i]] = values[i]  
return d
```

2

Why This is Wrong

The index is given directly. You don't need to search for it by looping through the list.

This approach adds unnecessary complexity: creating an empty dictionary, iterating through all indices, checking a condition, and adding to dictionary only when the condition matches.

3

Lesson Learned

When you have direct access to what you need, **use it directly**. Don't over-engineer the solution.

Another Common Mistake

The Incorrect Solution

```
return dict(zip(keys, values))
```

What This Creates

```
{  
    'apple': 10,  
    'banana': 20,  
    'cherry': 30  
}
```

Always read the problem carefully. Using `dict(zip(keys, values))` is correct Python, but it solves a different problem than what was asked.

✗ The Problem

`zip()` combines **all** elements from both lists into pairs.

BUT the question wants only ONE key-value pair.

The question asks for a dictionary with ONE key-value pair at a specific index, not all possible pairs.

Engineering Best Practices



Use Direct Indexing When Possible

If the position is already known, don't loop or search for it. Direct access is faster and clearer.



Keep Code Minimal

Less code means fewer opportunities for bugs and easier maintenance. Write only what's necessary.



Trust Built-in Features

Python's negative indexing and list access are reliable. Use them confidently without extra logic.

Logic Summary

The problem is simpler than it might first appear. You have two lists and an index. You need one dictionary with a single key-value pair.

The complete solution:

1. Access the key element using `keys[index]`
2. Access the value element using `values[index]`
3. Combine them into a dictionary: `{keys[index]: values[index]}`
4. Return the result